

CSE331

AUTOMATA AND COMPUTABILITY

Turing machines: encoding, proving decidability, proving undecidability

Summer 2026

Decidable vs Recognizable

- Let M be a TM

$$L = \{ w \mid \text{Machine says yes} \}$$

- M **decides** a language L if:

- If $w \in L$ then $M(w)$ accepts
- If $w \notin L$ then $M(w)$ **rejects**

} M always halts!

M
└─ (i) Decider
└─ (ii) Recognizer

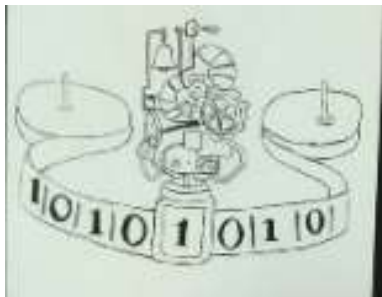
- M **recognizes** a language L if:

- If $w \in L$ then $M(w)$ accepts
- If $w \notin L$ then $M(w)$ **rejects or loops** (runs forever)

↗ Not halting

Decidable vs Recognizable

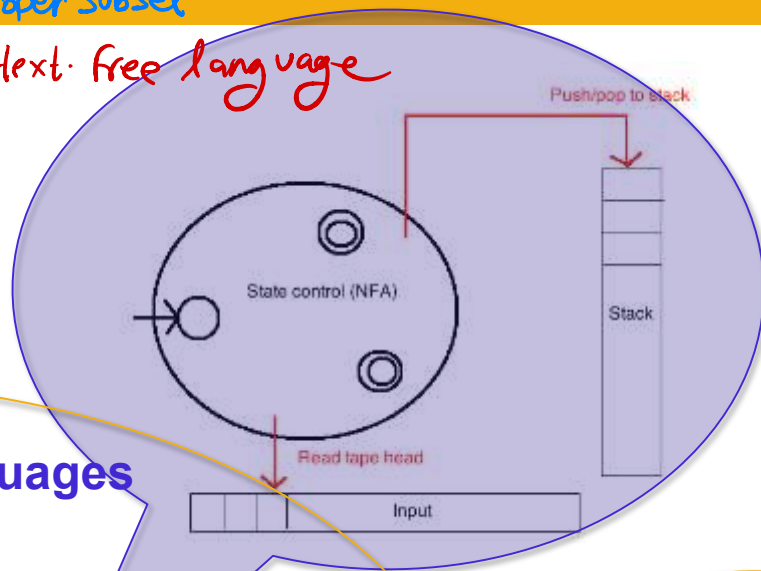
	M decides L	M recognizes L
$w \in L$	$M(w)$ accepts	$M(w)$ accepts
$w \notin L$	$M(w)$ rejects	$M(w)$ rejects or loops



Regular lang $\subsetneq$ Context-free language

$$\{0^n 1^n \mid n \geq 0\}$$

Proper Subset



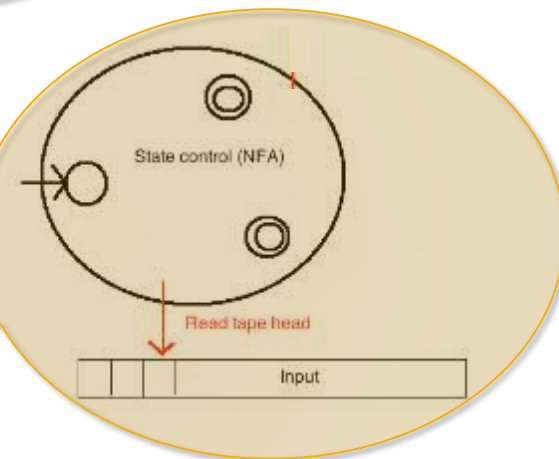
Turing recognizable languages

$w \notin L : M \text{ loops}$

Turing decidable languages

Context-free languages

Regular languages



$$\nearrow \{0^n 1^n \mid n \geq 0\}$$

Regular $\subsetneq$ Context free $\subsetneq$ Decidable $\subsetneq$ Recognizable.

$$A \subsetneq B$$

- (i) Every element of A is also in B
- (ii) But B contains at least one element that is not in A .

Today's learning goals

Sipser 3.1,3.2

- Use counting arguments to prove the existence of unrecognizable (undecidable) languages.
- Use diagonalization in a proof of undecidability.

DFA $\rightarrow \boxed{T_M}$

Sipser p. 159

Encoding input for TMs

- By definition, TM inputs are **strings**

- To define TM M:

"On input w ..."

1. ..
2. ..
3. ...

For inputs that aren't strings,
we have to **encode the object**
(represent it as a string) first

Notation:

$\langle O \rangle$ is the **string** that represents (encodes) the object O

$\langle O_1, \dots, O_n \rangle$ is the **single** string that represents the tuple of objects $O_1, \dots, O_n$

DFA $A \rightarrow$ Description of $A \rightarrow$ binary string $\langle A \rangle$

Example: encoding a DFA

- Think of **encoding** as a **text file** that encodes your data (in some format), that then you store in binary

$\langle A \rangle$ means the encoded description of A . It is like saving DFA in text file.

Example: encode a DFA $= (Q, \Sigma, \delta, q_0, F)$ using JSON

```
{
  "states": [1,2,3,4,5],           // Q={1,2,3,4,5}
  "alphabet": [0,1],               //  $\Sigma=\{0,1\}$ 
  "transition": [((1,0),2), (1,1), 3), ... ], //  $\delta(1,0) = 2, \delta(1,1) = 3, \dots$ 
  "initial_state": 1,              //  $q_0=1$ 
  "accept_states": [4,5],          // F={4,5}
}
```

Computational problems

$\langle D \rangle \rightarrow TM$

A computational problem is **decidable** iff the language encoding the problem instances is decidable

$$L_P = \{ \langle I \rangle \mid I \text{ is a YES-instance of problem } P \}$$

The problem P is decidable exactly when a TM can decide L_P .
(Accept every YES instance and reject NO instance, and always halts)

Undecidable?

- There are many ways to prove that a problem *is* decidable.
- How do we find (and prove) that a problem *is not* decidable?

Decidable : Exhibit one decider.

Undecidable : Prove that every proposed decider must fail.

A_{TM}

$$L_P = \{ \langle I \rangle \mid I \text{ is a YES-instance of problem } P \}$$

$w \in L(B)$

Recall $A_{DFA} = \{ \langle B, w \rangle \mid B \text{ is a DFA and } w \text{ is in } L(B) \}$

Decider for this set simulates arbitrary DFA

Recognizable, not decidable. This alone does not prove undecidability.

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$$

Decider for this set simulates arbitrary TMs ???

Prove that no
Smart decider
can exist.

What M does	Simulator does ^{$\rightarrow TM$}
Accept	Accept
Rejects	Rejects.
Loops	loop forever.

A_{TM}

→ "0/00/00/"

$$A_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$$

Define the TM $N =$ "On input $\langle M, w \rangle$:

1. Simulate M on w .
2. If M accepts, accept. If M rejects, reject."

High level
Description.

↓
N accepts

↓
N rejects

$$l(n) = A_{T_M}$$

If $w \in M$, M accepts, N accepts.

If $w \notin M$, M reject, N reject

IF $w \notin M$, M loop, N loop

A_{TM}

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$$

Define the TM $N =$ "On input $\langle M, w \rangle$:

1. Simulate M on w .
2. If M accepts, accept. If M rejects, reject."

What is $L(N)$?

A_{TM}

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$$

Define the TM N = "On input $\langle M, w \rangle$:

1. Simulate M on w .
2. If M accepts, accept. If M rejects, reject."

Does N decide A_{TM} ? *N recognizes A_{TM} , but does not decide it.*

A_{TM}

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$$

Define the TM $N =$ "On input $\langle M, w \rangle$:

1. Simulate M on w .
2. If M accepts, accept. If M rejects, reject."

Conclusion: A_{TM} is Turing-recognizable.

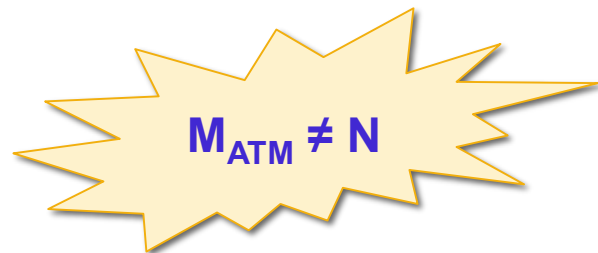
Is it decidable? *Does any decider for A_{TM} exist?*

Diagonalization proof: A_{TM} not decidable Sipser 4.11

Assume, **towards a contradiction**, that it is.

Call M_{ATM} the decider for A_{TM} :

↳ *fantasy machine*



For every TM M and every string w ,

- Computation of M_{ATM} on $\langle M, w \rangle$ halts and accepts if w is in $L(M)$.
- Computation of M_{ATM} on $\langle M, w \rangle$ halts and rejects if w is not in $L(M)$.

$\langle M, w \rangle$ = encoding of Machine M + input w
 M : The complete description of TM (states, transitions ...)
 w : Particular input string given to M .

Diagonalization proof: A_{TM} not decidable Sipser 4.11

Assume, towards a contradiction, that M_{ATM} decides A_{TM}

Define the TM $D =$ "On input $\langle M \rangle$:

M accept $\langle M \rangle$ | D rejects $\langle M \rangle$
 M rejects $\langle M \rangle$ | D accept $\langle M \rangle$

1. Run M_{ATM} on $\langle M, \langle M \rangle \rangle$.
2. If M_{ATM} accepts, reject; if M_{ATM} rejects, accept."

$M = D$

M_{ATM} on $\langle M, \langle M \rangle \rangle$

Does machine M accept its own description $\langle M \rangle$

Diagonalization proof: A_{TM} not decidable *Sipser 4.11*

Assume, towards a contradiction, that M_{ATM} decides A_{TM}

Define the TM $D =$ "On input $\langle M \rangle$:

1. Run M_{ATM} on $\langle M, \langle M \rangle \rangle$.
2. If M_{ATM} accepts, reject; if M_{ATM} rejects, accept."

Consider **running D on input $\langle D \rangle$** . Because D is a decider:

- **either computation halts and accepts ...**
- **or computation halts and rejects ...**

Question to M_{ATM} : Will D accept $\langle D \rangle$?

Diagonalization proof: A_{TM} not decidable Sipser 4.11

Assume, towards a contradiction, that M_{ATM} decides A_{TM} .

Because D is a TM, its encoding $\langle D \rangle$ can be given itself as input.

Define the TM $D =$ "On input $\langle D \rangle$:

1. Run M_{ATM} on $\langle D \rangle$.
2. If M_{ATM} accepts, reject; if M_{ATM} rejects, accept."

M_{ATM} says yes $\Rightarrow D$ says no
 M_{ATM} says no $\Rightarrow D$ says yes.

- If $D(\langle D \rangle)$ **accepts**, then $M_{ATM}(\langle D \rangle, \langle D \rangle)$ **rejects**, which means $D(\langle D \rangle)$ should have **rejected**.
- If $D(\langle D \rangle)$ **rejects**, then $M_{ATM}(\langle D \rangle, \langle D \rangle)$ **accepts**, which means $D(\langle D \rangle)$ should have **accepted**.
- Either way, we reached a **contradiction**.

A_{TM} is undecidable.

Diagonalization proof: A_{TM} not decidable Sinzer 4.11

Assume, for

Diagonalization???

Def

Self-reference

1. R

2. If

"Is $\langle D \rangle$ an element of $L(D)$?"

Consider **running on input $\langle D \rangle$** . Because D is a decider:

- **either computation halts and accepts ...**
- **or computation halts and rejects ...**

What does "diagonalization" mean?

Machine

$\langle M_1 \rangle$

$\langle M_2 \rangle$

$\langle M_3 \rangle$

M_1

Does M_1 accept $\langle M_1 \rangle$

M_2

Does M_2 accept $\langle M_2 \rangle$

M_3

Does M_3 accept $\langle M_3 \rangle$

D reverses every diagonal answer:

(i) If M accept $\langle M \rangle$, D rejects $\langle M \rangle$

(ii) Otherwise, D accepts $\langle M \rangle$

Finally, use $M = D$

$\langle D \rangle \in L(D)$

But D's rule gives: D accepts $\langle D \rangle \iff D$ does not accept $\langle D \rangle$

Why is this called diagonalization?

- Let $M_1, M_2, \dots$ be an enumeration of **all TMs**
- Let $w_1, w_2, \dots$ be an enumeration of **all inputs**
- Construct table: $T[i, j] = \begin{cases} 1 & \text{if } M_i(w_j) \text{ accepts} \\ 0 & \text{otherwise} \end{cases}$

	w_1	w_2	w_3	...
M_1	0	0	1	
M_2	1	0	1	
M_3	0	0	1	
...				

Why is this called diagonalization?

	w_1	w_2	w_3	...
M_1	0	0	1	
M_2	1	0	1	
M_3	0	0	1	
...				

*M_1 accepts w_3
But not w_1 and w_2*

- Rows of table describe **all recognizable languages**
- Any other row (=language) **cannot be recognizable**
- We will construct such a language

This new row differs from M_i at $\langle M_i \rangle$

Why is this called diagonalization?

- Next idea: specialize only to inputs $\langle M_i \rangle$

Construct a new row by flipping the Diagonal:

Diagonal: 0 0 1
New row: 1 1 0

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	...
M_1	0	0	1	
M_2	1	0	1	
M_3	0	0	1	
...				

Construct a Non recognizable lang Δ

D 1 1 0 = It cannot be any row in the table.

- Suffices to construct row (=language) that differs from any row on some of these inputs

So no TM recognizes this new lang.

This new lang. differs from M_i at $\langle M_i \rangle$

Why is this called diagonalization?

- Consider the **diagonal**: running M_i on $\langle M_i \rangle$; then **flip it**

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	...
M_1	0	0	1	
M_2	1	0	1	
M_3	0	0	1	
...				

Diagram illustrating the diagonalization process. A red diagonal line runs from the top-left cell (0) to the bottom-right cell (1). Red dashed lines connect the diagonal elements to their corresponding machine and input strings: $M_1(\langle M_1 \rangle)$ and $M_2(\langle M_2 \rangle)$. Below the table, the diagonal elements are listed: 0, 1, 1, 0, —.

- $L = \{\langle M \rangle : M \text{ doesn't accept } \langle M \rangle\}$
- L cannot be recognized by any M_i – they disagree on input $\langle M_i \rangle$
- Since we consider all TMs, L must be **un-recognizable**

A_{TM}

- Recognizable
- Not decidable

Two programs running Together

- ① Yes-detector: Recognize A_{TM} . Eventually report if M accept w .
- ② No-detector: Recognize the complement. Eventually reports if M does not accept w .

Fact: A language is decidable iff it and its complement are both recognizable.

$\overline{A_{TM}}$ is unrecognizable.

Corollary: The complement of A_{TM} is **unrecognizable**

$$A_{TM}^C = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is not in } L(M) \}$$